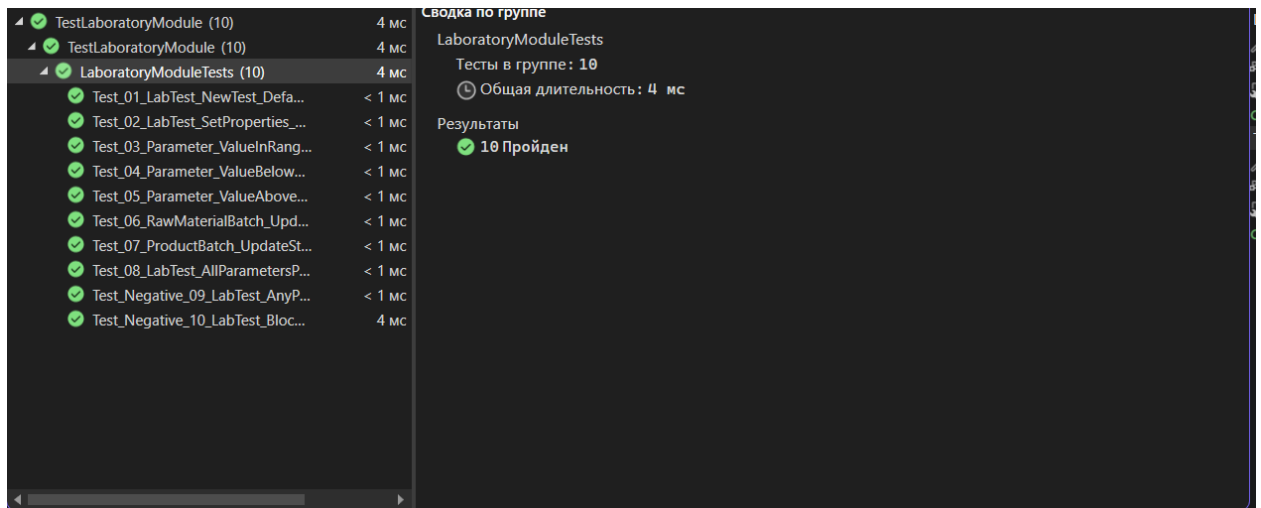




```
using Microsoft.VisualStudio.TestTools.UnitTesting;
```

```
using System.Collections.Generic;
```

```
namespace TestLaboratoryModule
```

```
{
```

```
    [TestClass]
```

```
    public class LaboratoryModuleTests
```

```
    {
```

```
        // ===== ТЕСТОВЫЕ МОДЕЛИ =====
```

```
        public class TestLabTest
```

```
        {
```

```
            public long Id { get; set; }
```

```
            public string TestNumber { get; set; }
```

```
            public string Status { get; set; }
```

```
            public string Decision { get; set; }
```

```
            public string DecisionReason { get; set; }
```

```
        }
```

```
        public class TestParameter
```

```
        {
```

```
            public string Name { get; set; }
```

```
            public decimal? MeasuredValue { get; set; }
```

```
            public decimal? StandardMin { get; set; }
```

```
    public decimal? StandardMax { get; set; }

    public bool IsPass { get; set; }
}
```

```
public class TestRawMaterialBatch
{
    public long Id { get; set; }

    public string BatchNumber { get; set; }

    public string Status { get; set; }
}
```

```
public class TestProductBatch
{
    public long Id { get; set; }

    public string BatchNumber { get; set; }

    public string Status { get; set; }
}
```

```
// ===== ПОЗИТИВНЫЕ ТЕСТЫ (8 шт.) =====
```

```
[TestMethod]
public void Test_01_LabTest_NewTest_DefaultValues()
{
    var test = new TestLabTest();

    Assert.AreEqual(0, test.Id);
    Assert.IsNull(test.TestNumber);
    Assert.IsNull(test.Status);
    Assert.IsNull(test.Decision);
}
```

```
[TestMethod]
```

```

public void Test_02_LabTest_SetProperties_ValuesSaved()
{
    var test = new TestLabTest
    {
        Id = 100,
        TestNumber = "LT-001",
        Status = "completed",
        Decision = "approved"
    };

    Assert.AreEqual(100, test.Id);
    Assert.AreEqual("LT-001", test.TestNumber);
    Assert.AreEqual("completed", test.Status);
    Assert.AreEqual("approved", test.Decision);
}

[TestMethod]
public void Test_03_Parameter_ValueInRange_ShouldPass()
{
    var param = new TestParameter
    {
        MeasuredValue = 97,
        StandardMin = 95,
        StandardMax = 100
    };

    bool isPass = param.MeasuredValue >= param.StandardMin &&
        param.MeasuredValue <= param.StandardMax;

    Assert.IsTrue(isPass);
}

```

[TestMethod]

public void Test_04_Parameter_ValueBelowMin_ShouldFail()

{

var param = new TestParameter

{

MeasuredValue = 90,

StandardMin = 95,

StandardMax = 100

};

bool isPass = param.MeasuredValue >= param.StandardMin &&

param.MeasuredValue <= param.StandardMax;

Assert.IsFalse(isPass);

}

[TestMethod]

public void Test_05_Parameter_ValueAboveMax_ShouldFail()

{

var param = new TestParameter

{

MeasuredValue = 105,

StandardMin = 95,

StandardMax = 100

};

bool isPass = param.MeasuredValue >= param.StandardMin &&

param.MeasuredValue <= param.StandardMax;

Assert.IsFalse(isPass);

}

[TestMethod]

```
public void Test_06_RawMaterialBatch_UpdateStatus_Changes()
```

```
{
```

```
    var batch = new TestRawMaterialBatch { Status = "pending" };
```

```
    batch.Status = "approved";
```

```
    Assert.AreEqual("approved", batch.Status);
```

```
}
```

[TestMethod]

```
public void Test_07_ProductBatch_UpdateStatus_Changes()
```

```
{
```

```
    var batch = new TestProductBatch { Status = "quality_control" };
```

```
    batch.Status = "approved";
```

```
    Assert.AreEqual("approved", batch.Status);
```

```
}
```

[TestMethod]

```
public void Test_08_LabTest_AllParametersPass_DecisionApproved()
```

```
{
```

```
    var parameters = new List<TestParameter>
```

```
    {
```

```
        new TestParameter { MeasuredValue = 97, StandardMin = 95, StandardMax = 100 },
```

```
        new TestParameter { MeasuredValue = 98, StandardMin = 95, StandardMax = 100 }
```

```
    };
```

```
    bool allPass = true;
```

```
    foreach (var p in parameters)
```

```
    {
```

```

        bool isPass = p.MeasuredValue >= p.StandardMin && p.MeasuredValue <= p.StandardMax;
        if (!isPass) allPass = false;
    }

    string decision = allPass ? "approved" : "blocked";

    Assert.AreEqual("approved", decision);
}

// ===== НЕГАТИВНЫЕ ТЕСТЫ (2 шт. - 20%) =====

[TestMethod]
public void Test_Negative_09_LabTest_AnyParameterFails_DecisionBlocked()
{
    var parameters = new List<TestParameter>
    {
        new TestParameter { MeasuredValue = 97, StandardMin = 95, StandardMax = 100 },
        new TestParameter { MeasuredValue = 85, StandardMin = 95, StandardMax = 100 }
    };

    bool allPass = true;
    foreach (var p in parameters)
    {
        bool isPass = p.MeasuredValue >= p.StandardMin && p.MeasuredValue <= p.StandardMax;
        if (!isPass) allPass = false;
    }

    string decision = allPass ? "approved" : "blocked";

    Assert.AreEqual("blocked", decision, "Есть отклонение → решение blocked");
}

```

```
[TestMethod]
public void Test_Negative_10_LabTest_BlockedWithoutReason_ShouldBeInvalid()
{
    string decision = "blocked";
    string reason = "";

    bool isValid = !(decision == "blocked" && string.IsNullOrEmpty(reason));

    Assert.IsFalse(isValid, "Блокировка без причины невалидна");
}
}
```